State-Aware Runtime for Long-Horizon LLM Agents: A Conceptual Framework and Research Agenda

Xiwei Chen
Independent Researcher
962557130s@gmail.com

May 2026

Abstract

Long-horizon LLM agents are increasingly expected to operate across extended interactions, evolving tasks, tool calls, memory updates, and multi-step plans. However, many failures in such systems are not adequately explained by single-turn reasoning errors or insufficient model capability. Instead, they arise from unstable state maintenance, uncontrolled memory injection, protocol drift, tool-mediated side effects, and missing recovery mechanisms. This paper defines **State-Aware Runtime** as the transaction-governance layer that separates model generation from canonical state, memory operations, validation, commit/rollback, and audit traces during execution. Its lifecycle includes boot-time initialization, session resume, baseline recovery, bounded state-view construction, proposal validation, commit, rollback or compensation, and audit. Stronger models may reduce the frequency of invalid proposals, but they do not remove the need for durable state, audit trails, permission boundaries, and side-effect governance when agent actions persist beyond a single model context. We provide a structured conceptual review of work on agent memory, tool use, long-context modeling, self-reflection, generative agents, workflow orchestration, and monitoring, and reorganize these strands around the problem of long-horizon runtime reliability. The central claim is that future agent reliability will depend less on prompt accumulation and more on explicit runtime systems that manage what the model is allowed to know, change, commit, forget, and recover. We conclude with a taxonomy of long-horizon failures and a research agenda for auditable, recoverable, and state-aware agent infrastructure.

1 Introduction

Large language models are increasingly embedded in agentic systems that interact with users, tools, files, environments, and external memory over long periods of time. These systems are no longer simple text generators. They are expected to maintain goals, update internal and external state, preserve user preferences, follow evolving constraints, operate tools, and recover from failures across many turns or episodes.

A common response to this challenge has been to expand the prompt: longer system instructions, larger context windows, richer memory summaries, more detailed tool descriptions, and increasingly complex behavioral rules. While such techniques can improve short-term controllability, they do not fully address the core reliability problem. Long-horizon agents fail not only because they lack information, but because they lack an explicit runtime layer that governs state, memory, validation, and recovery.

This paper develops a conceptual framework for long-horizon LLM agent reliability through the lens of **State-Aware Runtime**. The framework treats long-running agent execution as a lifecycle: boot or session resume establishes a known-good baseline; ordinary execution constructs bounded state views, receives model proposals, validates them, and commits only governed transitions; recovery restores, compensates, or escalates after failure.

https://doi.org/10.33774/coe-2026-vt9t2 Content not peer-reviewed by Cambridge University Press. **License:** CC BY 4.0

The motivation is simple: a model call should not be treated as the agent itself. In a robust agent architecture, the model acts as an actor, planner, narrator, or proposal generator. The agent system must separately maintain canonical state, memory stores, transition rules, validation gates, tool permissions, rollback mechanisms, and audit logs. Without this separation, long-running systems are vulnerable to silent state corruption, memory pollution, rule drift, and irreversible tool-mediated errors.

This paper makes three contributions:

1. It defines State-Aware Runtime as a lifecycle framework that externalizes state-transition governance, validation, memory operations, recovery, and audit from the model prompt.
2. It separates boot/resume/recovery, bounded state views, proposal surfaces, validator coverage, commit discipline, rollback or compensation, and handoff control as analyzable runtime phases.
3. It proposes a failure taxonomy, evaluation dimensions, and research agenda for long-horizon agent reliability, connecting prior work on memory, tool use, agent benchmarks, self-reflection, monitoring, and workflow orchestration.

This is a conceptual framework paper and research agenda rather than an exhaustive literature catalog. Future work may instantiate this framework through controlled evaluations, long-form generation systems, interactive agents, and protocol-fidelity benchmarks.

2 Scope and Conceptual Review Method

This paper is a structured conceptual review and position paper rather than a statistical meta-analysis or exhaustive catalog. Its goal is to reorganize adjacent work around the reliability problem of long-horizon agent runtime state. The review is guided by four questions:

1. What components have prior systems used to extend LLMs into agents?
2. How do those systems represent, retrieve, validate, and update state?
3. Which failures remain poorly explained when evaluation focuses only on final outputs?
4. What operational abstractions are needed to make long-horizon agents auditable, recoverable, and rollback-capable?

The literature was selected from six source classes: peer-reviewed and preprint work on LLM agents, memory-augmented models, tool use, multi-agent workflows, long-context modeling, and agent benchmarks; official engineering documentation from major model providers; selected systems work on runtime verification, state machines, logging, and transaction processing; formal verification and proof-assistant work relevant to verifier-in-the-loop agents; coding-agent benchmarks and software-engineering agent papers; and safety work on process supervision, monitoring, and chain-of-thought faithfulness.

The review includes work when it contributes at least one of the following: an agent architecture, a memory or context-management mechanism, a tool-use or workflow pattern, an evaluation environment for multi-step agents, a monitoring or supervision mechanism, or a systems abstraction relevant to state transition, rollback, or audit. It excludes papers that only report single-turn benchmark improvements without implications for agent runtime behavior.

Each work is analyzed along a common set of runtime dimensions:

https://doi.org/10.33774/coe-2026-vt9t2 Content not peer-reviewed by Cambridge University Press. **License:** CC BY 4.0

Dimension	Question
State object	What persistent state, if any, is represented outside the model context?
State view	What subset of state is exposed to the model at a given step?
Proposal surface	What actions, plans, text, tool calls, or memory writes can the model propose?
Validation	Which schemas, tests, verifiers, policies, or human checks constrain the proposal?
Commit discipline	How does a proposal become durable state?
Recovery	What happens when validation fails or a tool action has bad effects?
Auditability	Can the execution trace explain why the system changed state?

This methodology intentionally favors runtime interpretability over leaderboard comparison. It asks not only whether a system works, but where state lives, who may change it, how invalid transitions are blocked, and whether a failed trajectory can be reconstructed.

3 Framework Overview

The framework can be summarized as a set of runtime objects rather than as a single prescribed implementation. A system may instantiate these objects with files, databases, workflow graphs, checkpoint stores, event logs, schemas, tests, human review queues, or platform SDK primitives.

3.1 Framework canvas

Component	Runtime role	Input	Output	Failure if absent
Boot / resume / recovery phase	Establishes or restores a known-good baseline.	Artifacts, checkpoints, invariant set, environment checks.	Known-good operating baseline.	Session drift or unsafe restart.
Canonical state	Authoritative state outside the model context.	Committed facts, task state, user preferences, tool state.	Versioned state object.	State becomes narrative residue.
State view	Bounded model-visible slice of state.	Canonical state, current observation, context budget.	Prompt or context packet.	Attention pollution or missing constraints.
Proposal surface	Defines what the model may propose.	Allowed actions, schemas, tool contracts.	Structured proposal or state diff.	Unconstrained natural-language edits.
Validator or gate	Checks proposals before commit.	Proposal, invariants, policies, executable feedback.	Accept, reject, repair, retry, rollback, or escalate.	Invalid commits.
Commit log	Durable history of accepted and rejected transitions.	Decision, state diff, validator result, actor identity.	Trace and rollback pointer.	Audit failure.
Recovery policy	Defines how failures are handled.	Failed proposal, bad commit, broken environment, side effect.	Restored state, compensation, safe mode, or escalation.	Unrecoverable drift.

This canvas is intentionally implementation-neutral. It is meant to let different agent frameworks be compared by their state discipline: what state is authoritative, how much of it the model sees, what the model may change, what is checked, and what becomes durable.

https://doi.org/10.33774/coe-2026-vt9t2 Content not peer-reviewed by Cambridge University Press. **License:** CC BY 4.0

3.2 Completeness criteria

An implementation is **boot-complete** for a domain if it can answer the following at cold start, session resume, or baseline recovery:

1. What state is authoritative?
2. Which artifacts define current progress?
3. Which invariants and tool boundaries must be loaded?
4. Which checks confirm that the environment is usable?
5. Which checkpoint or baseline can be restored after failure?

An implementation is **runtime-complete** for a domain if it can answer the following during ordinary execution:

1. Which state view is visible to the model?
2. Which proposals are allowed?
3. Which validators or gates are applied?
4. What conditions allow a proposal to commit?
5. Does the commit produce a state diff and rollback pointer?
6. Does failure trigger reject, repair, retry, rollback, compensation, or escalation?
7. Can the trace explain why state changed?

These criteria are not binary claims about all possible tasks. They are domain-relative. A creative writing assistant, a coding agent, and a medical workflow agent require different completeness thresholds because their risks and available validators differ.

3.3 Reference architecture

A minimal engineering layout can be expressed as:

```
boot_resume_recovery/
  state_schema.yaml
  invariant_set.yaml
  artifact_manifest.yaml
  tool_bootstrap.yaml
  resume_policy.yaml
  recovery_policy.yaml

runtime/
  state_manager
  context_allocator
  memory_governor
  proposal_parser
  validator_stack
  side_effect_controller
  commit_manager
  rollback_manager
  audit_logger
  handoff_controller
```


https://doi.org/10.33774/coe-2026-vt9t2 Content not peer-reviewed by Cambridge University Press. **License:** CC BY 4.0

The same architecture can expose a small set of machine-checkable objects:

```
state_object = {
  id,
  version,
  authority,
  fields,
  invariants,
  last_commit_hash
}

proposal = {
  actor,
  action_type,
  target_state_field,
  evidence,
  proposed_diff,
  side_effect_class
}

validator_result = {
  syntax,
  schema,
  invariant,
  executable,
  policy,
  decision,
  rollback_pointer
}
```

This reference architecture is not a required file format. It states the minimum information that must exist somewhere if a long-horizon agent is to be audited, resumed, and repaired.

3.4 Runtime profiles

State-aware runtime is a design space, not a requirement that every agent use heavyweight governance. A system can choose a profile according to risk, interaction speed, and validator availability:

Profile	**Suitable use**	**Required mechanisms**
Lightweight state pins	Low-risk chat, real-time NPCs, creative assistance.	Key invariants, small state pins, simple schema checks.
Transactional runtime	Coding agents, long-form generation, database-like workflows.	State diffs, proposal validation, commit log, rollback pointers.
Verifier-in-the-loop runtime	Proof, science, web navigation, coding benchmarks.	Executable validators, retry policy, trace preservation.
Human-escalated runtime	High-stakes medical, legal, financial, or safety-sensitive actions.	Human approval, conservative escalation, side-effect control.
Multi-agent runtime	Research swarms, workflow agents, specialist handoffs.	Handoff packets, state isolation, global commit rule, conflict handling.

https://doi.org/10.33774/coe-2026-vt9t2 Content not peer-reviewed by Cambridge University Press. **License:** CC BY 4.0

This profile view addresses the model-capability counterargument. Stronger models can reduce the frequency of bad proposals, but they do not remove the need for external persistence, audit, permissions, side-effect governance, or compliance evidence when state changes matter.

3.5 Non-goals and boundary conditions

State-Aware Runtime does not guarantee that the model becomes more capable. It does not solve every semantic or normative judgment, replace memory systems, or make all tasks fully automatable. It is also not synonymous with workflow orchestration: a workflow can route steps correctly while still lacking canonical state, validation, commit semantics, or recovery policy.

The framework is most useful when a task has durable state, identifiable transition boundaries, meaningful validators, or consequential side effects. It is less useful for one-off low-risk conversations where no external state is committed.

4 Background: From Model Capability to Runtime Reliability

4.1 The shift from single-turn reasoning to long-horizon operation

Early evaluations of LLMs often focused on single-turn tasks: question answering, summarization, translation, code generation, mathematical reasoning, and instruction following. In these settings, the main object of evaluation is the output of a single model call or a short dialogue.

Agentic systems change the evaluation target. A long-horizon agent is not only judged by whether one response is correct, but by whether it can preserve goals, maintain state, call tools safely, update memory appropriately, and recover from errors across extended interactions. The relevant unit of analysis becomes the trajectory: a sequence of observations, decisions, memory operations, state updates, tool calls, validations, and commits.

This shift makes reliability harder to measure. A system may perform well in isolated turns while gradually corrupting its state. It may generate plausible responses while losing track of earlier commitments. It may pass a local schema check while violating a global invariant. It may retrieve a true memory that is irrelevant or harmful in the current context. These failures are runtime failures, not merely language modeling failures.

4.2 Context expansion is not sufficient

Longer context windows allow more information to be placed before the model, but they do not guarantee stable access, correct prioritization, or safe execution. A larger context window can even intensify the problem by encouraging prompt accumulation: old rules, outdated summaries, stale constraints, and irrelevant memories are injected together into the same model call.

The result is often a fragile form of control. Rules that should be explicit runtime constraints become natural-language suggestions. State that should be canonical becomes narrative residue. Memory that should be selectively retrieved becomes an unfiltered context bundle. Tool permissions that should be enforced by the system become instructions that the model may or may not follow.

The long-horizon reliability problem therefore cannot be solved by context length alone. It requires a runtime architecture that determines which information is loaded, which state is authoritative, which actions are valid, which updates can be committed, and how failures are detected and repaired.

4.3 Runtime and model capability are complementary

This argument does not deny model progress. Larger context windows, better long-context attention, stronger state tracking, and improved self-correction can all reduce the frequency of runtime failures. Model capability determines the ceiling of what an agent can propose.

https://doi.org/10.33774/coe-2026-vt9t2 Content not peer-reviewed by Cambridge University Press. **License:** CC BY 4.0

However, context capacity is not the same as effective attention, and internal state tracking is not the same as durable external state. Even a very strong model still needs external mechanisms when the system must persist across sessions, audit decisions, enforce permissions, recover from compaction, validate against an environment, or govern irreversible side effects. In that sense, model capability determines the upper bound, while runtime architecture determines how much of that capability is converted into reliable production behavior.

4.4 Prompt engineering versus harness engineering

Prompt engineering controls model behavior primarily through natural-language instructions. Harness engineering controls agent behavior through the external system that surrounds the model: schemas, validators, state machines, memory stores, tool interfaces, monitors, rollback procedures, and audit logs.

The distinction is not absolute. Prompts remain necessary. However, prompt-only control becomes increasingly brittle as agent tasks become longer, more interactive, and more consequential. A long-horizon agent requires a harness that can externalize constraints and make them executable.

In this view, the prompt is not the agent's operating system. It is one input to a larger runtime.

4.5 From harness composition to runtime state maintenance

Harness engineering answers an increasingly well-recognized static question: **what components surround an agent?** Typical answers include tools, memory, retrieval, sandboxes, evaluators, user interfaces, logs, and human approval gates. This question is necessary, but it is not sufficient for long-horizon reliability.

The harder dynamic question is: **how do these components jointly maintain a long-lived, stable, auditable, rollback-capable, and recoverable state?** A system may contain all familiar harness components and still fail if they do not share an explicit state discipline. A memory module may retrieve facts, a tool layer may execute actions, and a validator may check local syntax, while the overall system silently commits an invalid transition.

This paper uses **State-Aware Runtime** to name that dynamic layer. The runtime is responsible for state ownership, state views, transition authority, commit discipline, recovery procedures, and traceability across repeated model calls. The distinction can be summarized as follows:

Layer	Primary question	Typical object	Reliability gap
Prompt engineering	What should the model be told?	instructions and examples	rules remain advisory
Harness engineering	What components surround the model?	tools, memory, validators, sandboxes	components may be disconnected
State-aware runtime	How are transitions governed during execution?	state views, proposals, validators, commits, traces	state transitions become governable

The boot/resume/recovery phase is the boundary-triggered part of this runtime view. It establishes or restores a known-good operating baseline at cold start, session resume, compaction recovery, or disaster recovery. Ordinary State-Aware Runtime then governs the execution loop: state views, model proposals, validation, commit, rollback or compensation, and audit.

https://doi.org/10.33774/coe-2026-vt9t2 Content not peer-reviewed by Cambridge University Press. **License:** CC BY 4.0

5 Boot, Resume, and Baseline Recovery

5.1 Definition

The **boot/resume/recovery phase** initializes or restores the operating baseline from which ordinary agent execution can safely continue. The informal label **AI BIOS** can be read only as a shorthand for this phase, not as a second framework-level contribution: analytically, boot, session resume, and baseline recovery are lifecycle phases of State-Aware Runtime.

This phase answers a narrow set of questions:

- What state is authoritative at startup or resume?
- Which artifacts define current progress?
- Which invariants and tool boundaries must be loaded before action?
- Which environment checks confirm that tools and files are usable?
- Which checkpoint or baseline can be restored after failure?
- Which audit prerequisites must exist before ordinary execution continues?

Without this phase, each fresh model context must reconstruct task state, constraints, progress, and tool permissions from dialogue residue. That reconstruction is unstable because the LLM proposer has no durable private state between calls; it only sees the state view reconstructed for the current invocation.

5.2 Operational contract

A boot/resume/recovery contract can be specified as:

```
boot_resume_recovery = {
  state_schema,
  boot_state,
  invariant_set,
  artifact_manifest,
  tool_bootstrap,
  resume_policy,
  recovery_policy,
  audit_prerequisites
}
```

Each field has an operational role:

https://doi.org/10.33774/coe-2026-vt9t2 Content not peer-reviewed by Cambridge University Press. **License:** CC BY 4.0

Field	Operational meaning	Failure if absent
`state_schema`	Defines canonical variables and allowed state types.	State becomes narrative residue in dialogue history.
`boot_state`	Provides the minimal initial state for a session or episode.	The agent reconstructs identity and task state from unstable context.
`invariant_set`	Defines constraints that must hold across transitions.	Rules remain advisory and can drift across turns.
`artifact_manifest`	Lists durable files, databases, logs, checkpoints, or repositories that define recoverable state.	The next session cannot know which artifacts are authoritative.
`tool_bootstrap`	Defines tools that must be initialized or smoke-tested before work resumes.	The agent may build on a broken environment.
`resume_policy`	Defines what to read and verify after compaction or a new context window.	Fresh sessions repeat work or continue from a false state.
`recovery_policy`	Specifies how to restore a previous known-good baseline after corruption.	Detected failures cannot be repaired safely.
`audit_prerequisites`	Specifies the minimal logging and checkpoint substrate required for later reconstruction.	Failure analysis becomes anecdotal rather than traceable.

5.3 Boundary with ordinary execution

The boundary can be stated operationally:

Phase	Trigger	Responsibility	Not responsible for
Boot / resume / recovery phase	cold start, session resume, compaction recovery, baseline recovery	initialize artifacts, load invariant constraints, recover known-good state, run basic environment checks	deciding every ordinary state transition
Ordinary runtime execution	each model step or tool-mediated action	construct state views, allocate context, validate proposals, commit or roll back state transitions, audit execution	reconstructing the whole operating baseline from scratch

This distinction keeps the framework from collapsing into a general memory system. The boundary phase is episodic and recovery-triggered. Ordinary runtime execution is continuous and transaction-triggered.

5.4 Why this phase is not just classic checkpointing

Classical checkpoint and recovery systems usually restore a durable program state, then resume a deterministic or explicitly coded client. LLM agents are different: the proposer that acts after recovery is a stateless model invocation whose behavior depends on the reconstructed state view, retrieved memories, prompt-visible constraints, and tool observations. A rollback that restores files or database state but leaves polluted memory, stale summaries, or missing constraints in the next context can reproduce the same failure.

For this reason, baseline recovery in LLM agents must couple state restoration with context reconstruction, memory governance, tool revalidation, and audit replay. The state object alone is not enough; the runtime must also decide what the next model call is allowed to see and propose.

https://doi.org/10.33774/coe-2026-vt9t2 Content not peer-reviewed by Cambridge University Press. **License:** CC BY 4.0

6 State-Aware Runtime

6.1 Definition

We define **State-Aware Runtime** as the execution layer that manages the relationship between model-generated proposals and canonical system state.

A state-aware runtime separates four roles:

1. The model proposes actions, text, plans, or state updates.
2. The runtime validates these proposals against schemas and invariants.
3. The state manager commits valid updates and rejects invalid ones.
4. The audit layer records the full process for debugging and analysis.

In this architecture, the model is not allowed to directly overwrite the agent's canonical state. It can only propose changes through structured interfaces.

6.2 Core architecture

A generic state-aware runtime can be described as follows:

```
Observation / user input / environment event
        ->
State selector
        ->
Attention budget allocator
        ->
Memory retriever / just-in-time loader / compaction engine
        ->
Prompt/context assembly from a bounded state view
        ->
LLM actor or planner
        ->
Structured output proposal
        ->
Schema, policy, and executable validation
        ->
Invariant checks and policy gates
        ->
Commit / rollback / fallback
        ->
Audit trace and updated canonical state
```

This architecture converts agent behavior from unconstrained text continuation into a controlled state transition process.

The insertion of an attention budget allocator is not cosmetic. Context is a limited execution resource. A runtime must decide not only what to retrieve, but what not to load. Long-horizon systems therefore need explicit context governance: high-signal state summaries, durable notes outside the context window, pointers to external artifacts, just-in-time retrieval through tools, and compaction policies that preserve task-critical constraints while discarding stale interaction residue.

https://doi.org/10.33774/coe-2026-vt9t2 Content not peer-reviewed by Cambridge University Press. **License:** CC BY 4.0

6.3 Canonical state

Canonical state is the authoritative representation of the agent's current situation. It may include:

- task status;
- user preferences;
- environment state;
- active constraints;
- tool permissions;
- memory references;
- unresolved commitments;
- character or role state;
- planning state;
- safety or policy state.

The central design rule is that canonical state should be external to the model context. The model may read selected views of the state, but the runtime owns the state.

6.4 State representation design

Canonical state is not required to use one storage technology. Different domains need different representations:

Representation	**Strength**	**Risk**
JSON or typed records	Simple schemas, easy diffs, natural fit for tool arguments.	Weak relational semantics unless invariants are separately encoded.
Relational databases	Strong constraints, transactions, queryability.	Less natural for open-ended narrative or episodic memory.
Knowledge graphs	Explicit relations and provenance.	Updates and conflict resolution can become complex.
Vector stores	Useful similarity-based retrieval.	Similarity is not authority; retrieved items still need governance.
Event logs	Strong audit trail and replay semantics.	Requires projection logic to derive current state.
Hybrid stores	Separates durable state, episodic evidence, and retrieval.	Requires clear authority rules across stores.

The runtime should make representation boundaries explicit. For example, vector memory may provide candidate evidence, but a typed state object or event log should decide what is committed. Similarly, an action schema should declare whether an operation is idempotent, whether it has external side effects, which state fields it may modify, and which validators must pass before commit.

https://doi.org/10.33774/coe-2026-vt9t2 Content not peer-reviewed by Cambridge University Press. **License:** CC BY 4.0

6.5 Context and attention management

State-Aware Runtime must manage context as a scarce resource. A larger context window does not remove the need for selection because long contexts can dilute attention, increase latency, and make stale instructions harder to distinguish from active constraints. The runtime should therefore maintain a **context budget** alongside the state model.

A minimal context-management policy includes:

Component	Runtime role
Attention budget allocator	Assigns token budget across boot constraints, current goal, state view, retrieved memory, tool results, and user-visible response.
Compaction engine	Converts long interaction history into structured notes, progress files, or state diffs that can survive context resets.
Just-in-time retrieval	Stores pointers to files, database queries, web pages, logs, or memories and loads them only when needed.
Staleness filter	Suppresses obsolete summaries, abandoned plans, superseded rules, and failed hypotheses.
Constraint pinning	Ensures invariant rules remain visible or executable even when ordinary context is compacted.

This layer is where State-Aware Runtime differs from ordinary retrieval. Retrieval asks which information is similar or relevant. Context governance asks which information deserves scarce attention in the next model call, which information should remain external, and which information must be kept out to avoid context rot.

6.6 Structured actions

Structured actions define the set of operations the model may propose. Instead of allowing arbitrary natural-language state edits, the runtime can expose a controlled action registry.

Examples include:

- `update_goal`;
- `retrieve_memory`;
- `mark_task_complete`;
- `ask_clarification`;
- `call_tool`;
- `transition_scene`;
- `record_preference`;
- `defer_decision`;
- `enter_safe_mode`.

Structured actions reduce ambiguity and make validation easier. They also make traces more analyzable because each model output can be mapped to a finite set of state operations.

Structured generation and constrained decoding techniques can help implement this proposal surface by making invalid formats harder to produce [Willard and Louf, 2023]. They are still only one layer of control: a syntactically valid action can remain semantically invalid, unauthorized, or unsafe to commit.

For consequential tools, the action schema should also declare effect metadata: whether the action is read-only, reversible, compensatable, or irreversible; whether it is idempotent; whether

https://doi.org/10.33774/coe-2026-vt9t2 Content not peer-reviewed by Cambridge University Press. **License:** CC BY 4.0

it touches external systems; and whether human approval is required. This metadata lets the runtime distinguish a harmless draft edit from an email send, payment call, production database write, or public post.

6.7 Validation and gates

Validation occurs at multiple levels:

- syntax validation: whether the output is parseable;
- schema validation: whether required fields are present;
- type validation: whether values have the correct type;
- semantic validation: whether proposed actions are meaningful;
- invariant validation: whether global constraints are preserved;
- policy validation: whether actions are allowed;
- transition validation: whether the system can legally move from one state to another.

A common failure in agent systems is to check only syntax or schema compliance while ignoring semantic and state-level validity. A JSON object can be valid while still representing an invalid state transition.

Validation should not be treated as uniformly tractable. Different domains admit different levels of automatic checking:

Validation class	**Examples**	**Tractability**
Syntactic	JSON parse, field presence, enum membership	Usually automatic and cheap.
Type and schema	typed tool arguments, API contracts, state-diff schemas	Usually automatic when interfaces are well specified.
Executable	unit tests, compiler errors, browser automation checks, proof checkers, simulation rollouts	Highly valuable when the environment exposes ground truth, but can be slow or incomplete.
Invariant	access control, timeline constraints, accounting balance, branch preconditions	Tractable when invariants are formalized; difficult when they remain implicit.
Semantic	intent preservation, creative quality, narrative motivation, user satisfaction	Often partial, noisy, or human-assisted rather than fully automatic.
Social or normative	fairness, harm, appropriateness, value conflict	Requires policy, human review, or conservative escalation in high-stakes settings.

The runtime should also distinguish **hard gates** from **soft gates**:

Gate type	**Typical mechanisms**	**Runtime effect**
Hard gates	JSON schema, AST parser, type checker, file-system sandbox, tool permissions, access-control policy, deterministic invariants.	Block commit, prevent side effects, force rollback, or enter safe mode.
Soft gates	LLM-as-a-judge, process reward model, style classifier, narrative-quality critic, social-norm scorer.	Trigger retry, repair, warning, lower confidence, or human escalation.

https://doi.org/10.33774/coe-2026-vt9t2 Content not peer-reviewed by Cambridge University Press. **License:** CC BY 4.0

Hard gates protect the boundary between proposal and committed state. They are appropriate for parseability, permissions, protected files, irreversible actions, forbidden state transitions, and other properties where false acceptance is costly. Soft gates are useful when the property is semantic, aesthetic, social, or only partially formalized. A soft gate should usually not be the sole reason to commit or block a high-risk action; it should route the proposal into repair, retry, or review.

The runtime should prefer executable and environmental feedback whenever available. For coding, this means compilers, tests, static analysis, diffs, and CI. For web agents, it may include browser automation, DOM checks, screenshots, and task-specific end-to-end tests. For proof agents, it means proof checkers and dependency verification. LLM-based critique can be useful, but it should not be the only gate for consequential commits.

This also limits the claims of State-Aware Runtime. The framework does not assume that every semantic property can be perfectly verified. Instead, it requires the runtime to make validator coverage explicit: which properties are automatically checked, which are approximated, which require human review, and which remain unverified residual risk.

In practice, validation should usually be progressive. Cheap gates such as parsing, schema checks, permission checks, and invariant checks run first. More expensive gates such as tests, browser automation, proof checking, simulation, or human review run only when the proposal reaches a consequential transition boundary. This mirrors runtime-verification practice: safety properties that can be monitored should be monitored continuously, while ambiguous properties should be escalated rather than hidden behind an LLM self-critique.

Validation is also a security boundary. Tool results, retrieved memories, web pages, emails, and user-uploaded files can carry prompt-injection content or malicious instructions. A state-aware runtime should therefore apply least-privilege tool access, source labeling, taint tracking for untrusted observations, and policy gates before retrieved content is allowed to influence canonical state.

6.8 Commit and rollback

Commit and rollback mechanisms are central to long-horizon reliability. If a proposed action passes validation, the runtime commits it to canonical state. If it fails, the runtime rejects it, repairs it, retries under constrained conditions, or rolls back to the previous stable state.

This makes agent execution closer to transaction processing than ordinary dialogue generation. The model may be stochastic, but the state transition process should be controlled. Rollback applies primarily to internal or reversible state; external side effects require the side-effect controller described next.

6.9 Side-effect governance

Rollback is straightforward for internal state, files under version control, generated text, or sandboxed artifacts. It is not straightforward for irreversible external side effects. Once an agent sends an email, charges a card, posts publicly, changes a production database, or moves a physical actuator, the runtime cannot assume that restoring `S_t` restores the world.

State-Aware Runtime therefore needs a side-effect controller. At minimum, each action should be classified as:

Effect class	**Runtime policy**
Read-only	Log and allow under ordinary permission checks.
Reversible internal write	Allow after validation; attach rollback pointer.
Compensatable external action	Require preconditions and define a compensating action, such as refund, cancellation, or corrective notice.
Irreversible or high-risk action	Require dry-run, stronger validation, human approval, or safe-mode escalation before execution.

https://doi.org/10.33774/coe-2026-vt9t2 Content not peer-reviewed by Cambridge University Press. **License:** CC BY 4.0

This extends the transaction analogy toward long-running transactions and saga-like recovery [Garcia-Molina and Salem, 1987]. For external effects, the right recovery action is often not rollback but compensation: a new action that acknowledges the effect occurred and attempts to repair its consequences. The audit trace should record both the original side effect and any compensation attempt.

6.10 Audit traces

A state-aware runtime should preserve enough trace information to reconstruct why a state changed. An audit trace may include:

- input event;
- selected state view;
- retrieved memories;
- prompt template version;
- model and decoding parameters;
- raw model output;
- parsed action;
- validation results;
- committed state diff;
- rejected proposals;
- fallback events.

These traces support debugging, reproducibility, benchmark construction, and failure taxonomy development.

6.11 State transition model

State-Aware Runtime can be written as a controlled transition model. At step `t`, the runtime owns canonical state `S_t`. The model never receives `S_t` directly. Instead, the runtime constructs a bounded state view `V_t` under a context budget `B_t`:

```
V_t = f_view(S_t, O_t, M_t, B_t, policies)
```

The model then emits a proposal rather than a committed state update:

```
P_t ~ pi_theta(V_t, A_t)
```

The runtime validates the proposal:

```
D_t = G(P_t, S_t, I, E, policies)
```

where `A_t` is the allowed action schema, `I` is the invariant set, `E` is the available executable environment feedback, and `G` is the gate or validator function.

More explicitly, `S_t` denotes the authoritative canonical state at step `t`; `O_t` denotes the current observation or event; `M_t` denotes retrieved memory or evidence selected by the memory governor; `B_t` denotes the context or attention budget; `policies` denote safety, privacy, tool,

https://doi.org/10.33774/coe-2026-vt9t2 Content not peer-reviewed by Cambridge University Press. **License:** CC BY 4.0

and domain policies; `V_t` denotes the bounded state view exposed to the model; `A_t` denotes the action schema available at that step; `P_t` denotes a sampled model proposal from policy `pi_theta`; and `D_t` denotes the runtime decision. The notation is intentionally lightweight: it is a bridge between agent engineering and more formal treatments such as finite-state machines, temporal logic monitors, POMDPs, and transaction systems.

The model is stochastic and partially informed. `V_t` is only a projection of `S_t`, and `P_t` is one sample from a distribution rather than a deterministic command. This is why the validator must judge proposals against canonical state and executable feedback, not only against the prompt-visible view. Richer formalizations could model the runtime as a partially observable stochastic controller with monitored safety properties, but the minimal requirement here is simpler: no sampled proposal becomes durable state without a governed transition.

The decision `D_t` is one of several transition outcomes:

Decision	Runtime action	State result
accept	Apply the parsed proposal through a controlled commit function.	`S_{t+1} = apply(S_t, P_t)`.
repair	Ask for a constrained revision or run an automated repair step.	State remains `S_t` until a valid proposal exists.
retry	Re-run the model with a narrower state view or stricter schema.	State remains `S_t`; retry is logged.
reject	Discard the proposal and continue from the prior stable state.	State remains `S_t`; rejection is logged.
rollback	Restore a previous checkpoint after detecting a bad reversible commit.	State returns to `S_k` for some earlier stable checkpoint.
compensate	Execute a compensating action for a committed external side effect.	Canonical state records both the side effect and compensation attempt.
escalate	Route the decision to a human or higher-authority process.	State is frozen or placed in safe mode.

The commit log is part of the state transition model, not an afterthought. A minimal commit entry should include:

```
commit_entry = {
  previous_state_hash,
  state_view_id,
  proposal_id,
  validator_results,
  decision,
  state_diff,
  rollback_pointer,
  timestamp,
  actor_or_model_id
}
```

This framing clarifies the central distinction between ordinary workflow automation and state-aware runtime. A workflow may execute a sequence of steps; a state-aware runtime controls which proposed transitions become durable state and preserves enough evidence to reverse or explain them.

The same model can be expressed as pseudocode:

```
for each event O_t:
    V_t = f_view(S_t, O_t, M_t, B_t, policies)
```


https://doi.org/10.33774/coe-2026-vt9t2 Content not peer-reviewed by Cambridge University Press. **License:** CC BY 4.0

```
    P_t = model.propose(V_t, allowed_actions)
    D_t = gate(P_t, S_t, invariants, executable_feedback, policies)

    if D_t.accept:
        S_{t+1} = commit(S_t, P_t)
        log_commit(S_t, P_t, D_t, S_{t+1})
    elif D_t.rollback:
        S_{t+1} = restore(D_t.rollback_pointer)
        log_rollback(S_t, P_t, D_t, S_{t+1})
    elif D_t.compensate:
        C_t = compensate(D_t.compensation_plan)
        S_{t+1} = record_compensation(S_t, P_t, C_t)
        log_compensation(S_t, P_t, D_t, C_t, S_{t+1})
    else:
        S_{t+1} = S_t
        log_rejection_or_repair(S_t, P_t, D_t)
```

6.12 Multi-agent handoff protocol

Multi-agent systems require state discipline across agent boundaries. A handoff should not be treated as an informal summary passed from one model to another. It is a state transition with routing metadata, filtered context, and commit semantics.

A minimal handoff protocol includes:

```
handoff_packet = {
  source_agent,
  target_agent,
  reason,
  permitted_state_view,
  local_findings,
  proposed_global_updates,
  confidence_or_evidence,
  redactions,
  validator_results,
  rollback_pointer
}
```

The runtime should enforce three rules:

1. **State isolation:** exploratory context accumulated by a sub-agent should remain local unless explicitly summarized and validated.
2. **Handoff commit:** the receiving or supervising runtime should treat the handoff packet as a proposal, not as automatically committed global state.
3. **Context filtering:** only the state view needed by the receiving agent should be transferred; tool traces, failed hypotheses, and sensitive data should be filtered unless they are necessary evidence.

This maps directly onto platform handoff mechanisms where a triage agent delegates to a specialist agent using structured handoff inputs, callbacks, run context, and input filters. The State-Aware Runtime contribution is to interpret these handoffs as governable state transitions rather than merely routing conveniences.

https://doi.org/10.33774/coe-2026-vt9t2 Content not peer-reviewed by Cambridge University Press. **License:** CC BY 4.0

Concurrent multi-agent systems introduce additional distributed-systems concerns: race conditions, conflicting writes, stale reads, and inconsistent local views. A full treatment of consensus, CRDTs, locks, operational transformation, or isolation levels is beyond the scope of this paper. The runtime prerequisite is nevertheless clear: each global update should carry a state version or commit parent, and conflicting proposals should be resolved through explicit merge rules, validator arbitration, or human escalation rather than last-write-wins by accident.

7 Literature Landscape

This section organizes related work around runtime reliability rather than around model capability alone. State-Aware Runtime is best understood as a synthesis across several adjacent lines of work. The aim is not to claim that prior systems lack state, memory, tools, or monitors. The claim is narrower: these strands are rarely organized around the lifecycle of **committed state** as the primary object of reliability.

The landscape can be summarized as a runtime taxonomy:

https://doi.org/10.33774/coe-2026-vt9t2 Content not peer-reviewed by Cambridge University Press. **License:** CC BY 4.0

Research line	Representative work	Main contribution	Runtime gap addressed here
Externalized reasoning and acting	Chain-of-thought prompting, ReAct, Tree of Thoughts, Graph of Thoughts, decomposed prompting	Move intermediate reasoning or actions outside a single opaque answer.	Intermediate traces do not automatically become validated state transitions.
Generative agents	Generative Agents, Voyager, ALFWorld, WebShop, ScienceWorld	Embed LLMs in environments with memory, plans, and actions.	Believability or task success does not guarantee auditable committed state.
Memory-augmented agents	MemGPT, Generative Agents memory streams, MemoryBank, LongMem	Extend context through external or hierarchical memory.	Memory retrieval and memory writing require authority, conflict handling, and commit discipline.
Tool-use and API agents	Toolformer, Gorilla, ToolLLM, API-Bank, HuggingGPT	Teach or prompt models to select and call tools.	Tool calls can have side effects; the runtime must validate and govern their effects on state.
Workflow and multi-agent systems	AutoGen, CAMEL, MetaGPT, ChatDev	Compose agents, roles, messages, and tool loops into higher-level workflows.	Workflow steps may be well organized while persistent state remains weakly governed.
Long-context and retrieval systems	Transformer-XL, RETRO, RAG, REALM, Lost in the Middle studies	Increase or retrieve accessible information.	More accessible information does not imply stable state ownership or safe update behavior.
Self-reflection and repair	Reflexion, Self-Refine, critique loops	Use feedback to revise outputs or improve future attempts.	Reflection itself must be constrained by validators and state diffs.
Agent benchmarks	AgentBench, WebArena, OSWorld, GAIA, SWE-bench, ToolBench	Evaluate multi-step tool use, web navigation, coding, and assistant behavior.	Many benchmarks still emphasize success rates over state-transition diagnostics.
Monitoring and process supervision	Process supervision, chain-of-thought monitoring, runtime verification	Inspect intermediate reasoning, steps, or execution events.	Monitoring becomes stronger when tied to commit, rollback, and trace semantics.
Platform agent SDKs	OpenAI Agents SDK, Responses API, built-in tools, guardrails and observability	Productize tool orchestration, state, approvals, and monitoring primitives.	A research abstraction is still needed for comparing state discipline across platforms.

The same dimensions can be used to compare representative systems and system lines. This table is not a source-level defect audit; it is a framework mapping that shows which runtime functions are explicit, partial, or platform-dependent.

https://doi.org/10.33774/coe-2026-vt9t2 Content not peer-reviewed by Cambridge University Press. **License:** CC BY 4.0

System or line	Runtime state	Proposal and validation surface	Recovery and audit gap
ReAct-style agents	Mostly implicit trajectory state in the prompt.	Reasoning and actions checked by environment feedback when available.	Retry-like recovery; partial trace.
Generative Agents	Memory stream, plans, reflections.	Plans, reflections, actions, and dialogue checked mostly heuristically.	Believable behavior is not the same as transactional state.
MemGPT-style memory systems	Memory tiers and external store.	Memory read/write operations and active context management.	Stronger memory discipline, but validation and recovery remain domain-specific.
AutoGen / multi-agent chat frameworks	Conversation state and routed agent messages.	Messages, tool calls, and handoffs with configurable controls.	Commit discipline and recovery vary by implementation.
LangGraph-style state graphs	Graph state, threads, checkpoints.	Node outputs and graph transitions with user-defined logic.	Recovery improves when checkpoints and tracing are configured.
Coding agents and SWE-bench agents	Repository state, issue context, patches.	Patches, commands, tests, linters, execution, review.	Strong audit when diffs and logs persist; external side effects still need governance.
OpenHands-style software agents	Workspace, sandbox, command history, agent state.	Code edits, commands, browser or API actions, sandbox checks.	Platform-level logs exist, but commit policy depends on task setup.
Platform SDK agents	Tools, state, approvals, guardrails, traces.	Tool calls, handoffs, messages, guardrails, approvals.	Reliability guarantees remain platform- and configuration-dependent.

The mapping suggests why the proposed framework is not a replacement for existing systems. It is a diagnostic lens: it asks whether persistence, validation, recovery, and audit are governed as a coherent state-transition system.

For example, LangGraph's documentation treats checkpoints as persistent graph-state snapshots that enable memory, fault tolerance, human-in-the-loop workflows, and time travel [LangChain, 2026]. OpenHands presents a software-agent platform in which agents write code, use a command line, browse the web, run in sandboxed environments, coordinate with other agents, and connect to benchmarks [Wang et al., 2024]. These systems show that stateful harness primitives are already practical; the framework here asks how to compare their reliability guarantees in a common vocabulary.

7.1 Generative agents and simulated social environments

Generative agent systems demonstrate how LLMs can be embedded in simulated environments with memory, reflection, and planning [Park et al., 2023]. These systems highlight the importance of external memory and behavioral scheduling, but they also reveal the challenge of maintaining consistent state across long interactions.

From the perspective of state-aware runtime, the key question is not only whether agents appear believable, but whether their memories, plans, and state transitions remain auditable and controllable over time.

7.2 Memory-augmented LLM agents

Memory-augmented agents use episodic memory, semantic memory, vector retrieval, summaries, or structured stores to extend interaction history. MemGPT explicitly frames LLM memory management through an operating-system analogy, with tiers of context and external storage [Packer et al., 2023]. These approaches address context limitations, but they also introduce new

https://doi.org/10.33774/coe-2026-vt9t2 Content not peer-reviewed by Cambridge University Press. **License:** CC BY 4.0

failure modes: stale retrieval, irrelevant injection, false consolidation, memory overfitting, and conflict between retrieved memories and current state.

A state-aware runtime treats memory retrieval as an operation requiring authorization, filtering, and validation. The memory system should not merely maximize similarity; it should select information compatible with the current task and state.

7.3 Tool-use agents and workflow orchestration

Tool-use agents extend LLMs with APIs, search, code execution, databases, and external services. ReAct interleaves reasoning traces and actions [Yao et al., 2023], Toolformer studies API-use selection [Schick et al., 2023], and frameworks such as AutoGen organize multi-agent conversation and tool-mediated workflows [Wu et al., 2023]. Industrial guidance has similarly emphasized simple, composable agent patterns, clear tool interfaces, and ground-truth feedback from environments [Anthropic, 2024].

These systems make state management more important because tool calls can have side effects. The runtime must decide when a tool call is allowed, whether its result is trustworthy, how the result updates state, and whether the action can be undone.

7.4 Self-reflection, critique, and repair

Self-reflection and critique-based methods encourage models to evaluate and revise their own outputs. Reflexion stores verbal feedback for later attempts [Shinn et al., 2023], while Self-Refine uses iterative self-feedback to improve outputs [Madaan et al., 2023]. These methods can improve local performance, but they may also introduce instability when the critique itself modifies the task interpretation or changes the target structure.

A state-aware runtime can use reflection as one component, but it should not rely on unconstrained self-critique as the only repair mechanism. Repair should be grounded in explicit validators, state diffs, and failure categories.

7.5 Long-context modeling and retrieval

Long-context models and retrieval-augmented systems increase the amount of accessible information. However, long-context access does not guarantee stable attention, correct priority, or faithful use of constraints. The "Lost in the Middle" results show that models can underuse information depending on its position in long contexts [Liu et al., 2023a].

State-aware runtime shifts the problem from ‘how much context can be loaded'' to ‘which state and memory views should be loaded for this action, under which constraints, and with which validation."

7.6 Monitoring, evaluation, and agent benchmarks

Agent benchmarks increasingly evaluate multi-step tasks, tool use, planning, web navigation, coding, and environment interaction. AgentBench is an example of a multi-environment evaluation suite for LLM agents [Liu et al., 2023b]. Long-horizon runtime reliability requires additional process-level metrics: state consistency, valid transition rate, memory stability, recovery success, audit completeness, and failure localization.

Outcome accuracy alone is insufficient. A system may reach a correct final answer through invalid intermediate state updates, or may appear coherent while accumulating hidden state corruption.

https://doi.org/10.33774/coe-2026-vt9t2 Content not peer-reviewed by Cambridge University Press. **License:** CC BY 4.0

7.7 Context engineering and long-running harnesses

Recent engineering guidance has shifted from prompt wording toward context selection, maintenance, and long-running harness design. Anthropic describes context engineering as the curation and maintenance of the token set available during inference [Anthropic, 2025a], and discusses harness practices for long-running agents such as incremental progress, clean environment state, progress files, tests, commits, and revertable work states [Anthropic, 2025b; Anthropic, 2026a; Anthropic, 2026b]. OpenAI's Agents documentation similarly presents agents as systems involving tools, orchestration, state, approvals, guardrails, observability, and handoffs [OpenAI, 2026a; OpenAI, 2026b].

State-Aware Runtime can be read as a research abstraction over these engineering practices. It asks how context, tools, tests, logs, approvals, memory, and recovery should be coordinated through a common state-transition discipline.

The mapping is direct:

Industrial harness pattern	**Runtime interpretation**
Initializer agent	Boot/resume/recovery mechanism for durable artifacts and initial constraints.
Feature list or task manifest	State baseline plus executable progress inventory.
Progress file or structured notes	Session-resume artifact and compaction target.
`init.sh` or environment setup script	Tool bootstrap policy and baseline environment check.
Git commit history	Commit log, rollback pointer, and audit trace for code state.
Start-of-session "getting up to speed" routine	Session-resume routine for recovering task bearings.
Browser automation, tests, compiler output	Environment-executable validation gate.
Handoff input schema and input filters	Multi-agent state-view control and handoff commit boundary.

This mapping grounds the framework in implementation. Boot and recovery are not new file formats; they are roles already played by manifests, progress files, initialization scripts, and version-control history in mature long-running harnesses. State-Aware Runtime names the transition layer that uses these artifacts during execution.

7.8 Executable feedback and checkable intermediate artifacts

Some agent environments are easier to govern because they expose dense executable feedback. Coding agents can use compilers, tests, type errors, stack traces, diffs, continuous integration, and version control as process-level signals. Formal proof and scientific-search agents can similarly use proof checkers, verifiers, experiment logs, dependency graphs, and failed-attempt records.

The common pattern is not merely tool use. It is the insertion of checkable intermediate artifacts between model generation and accepted state change. Examples include structured actions between language and tools, state diffs between proposals and commits, proof objects between mathematical ideas and accepted results, and audit traces between final outputs and system decisions. This pattern supports the state-aware view because it prevents important transitions from occurring only as unconstrained text.

https://doi.org/10.33774/coe-2026-vt9t2 Content not peer-reviewed by Cambridge University Press. **License:** CC BY 4.0

7.9 Systems foundations for state-aware agents

The framework also draws on older systems ideas. Transaction processing clarifies why proposal and commit should be separated [Gray and Reuter, 1993]. Sagas and compensating transactions clarify why long-running or irreversible side effects cannot always be rolled back [Garcia-Molina and Salem, 1987]. Event sourcing and command-query separation clarify why append-only traces and derived views can be useful for audit and replay [Fowler, 2005]. The actor model clarifies why agents that exchange messages need explicit state ownership and concurrency boundaries [Hewitt et al., 1973]. POMDPs and Dec-POMDPs provide formal language for partial observability and multi-agent decision processes [Kaelbling et al., 1998; Oliehoek and Amato, 2016]. CRDTs illustrate one family of approaches for conflict handling in replicated state, though many agent states require semantic arbitration rather than automatic merges [Shapiro et al., 2011].

The LLM-specific delta is not that classical systems lack rollback, logs, or monitors. It is that the component proposing the next transition is a stochastic natural-language model call with no durable private state across calls. In a database transaction, rollback restores data while the client program remains a stable piece of code. In an LLM agent, rollback must also govern the next context reconstruction: which state view is shown, which memories are quarantined, which stale summaries are removed, which validator feedback is injected, and which tool observations remain authoritative. Restoring `S_t` while leaving a polluted memory store or obsolete progress summary can lead the next model call to reproduce the same invalid proposal.

This is why State-Aware Runtime couples classical mechanisms with model-facing controls. Natural-language proposals often mix command, rationale, evidence, and speculation in one artifact; validators are partial; retrieved context can be adversarial or stale; and the agent's future behavior depends on attention allocation, not only on restored data. The framework therefore treats rollback, resume, memory governance, and state-view construction as coupled operations rather than independent subsystems.

8 Positioning Against Adjacent Concepts

State-Aware Runtime is close to several existing ideas, but the emphasis differs.

https://doi.org/10.33774/coe-2026-vt9t2 Content not peer-reviewed by Cambridge University Press. **License:** CC BY 4.0

Adjacent concept	What it contributes	Difference from this paper
Externalization	Makes reasoning, actions, memories, or intermediate artifacts visible outside the model.	State-Aware Runtime asks which externalized artifacts are authoritative, valid, commit-ready, and rollbackable.
Harness engineering	Builds the surrounding components around a model: tools, memory, schemas, sandboxes, logs, and approvals.	Harness engineering names the component layer; State-Aware Runtime names the transition discipline that coordinates those components over time.
MemGPT and memory operating systems	Treat context as limited memory and use external storage plus read/write operations.	State-Aware Runtime includes memory policy, but also boot constraints, tool permissions, validators, recovery policy, and audit obligations.
Generative Agents	Demonstrate believable agents with memory, reflection, planning, and social behavior.	This paper focuses less on believable behavior and more on whether state changes are validated, committed, audited, and recoverable.
Agents SDKs and platform primitives	Provide production abstractions for models, tools, state, guardrails, approvals, tracing, and observability.	This paper abstracts across platforms to define the reliability object: canonical state and its governed transitions.
Workflow agents	Compose model calls, tool calls, roles, and handoffs into executable task flows.	A workflow can be successful locally while still lacking global state ownership, rollback policy, or durable audit semantics.
Runtime verification	Checks whether system executions satisfy specified properties.	State-Aware Runtime imports this spirit but adapts it to stochastic model proposals, memory injection, tool side effects, and human-in-the-loop review.

The key claim is therefore not that prior systems lack external memory, tools, monitors, or workflows. It is that these components need a shared state model. Without such a model, a system can look agentic while still allowing invalid proposals to become persistent facts.

9 Closest Runtime Systems and Differentiation

Recent work has begun to treat agent runtime reliability as a first-class systems problem, emphasizing explicit boundaries, logs, validators, recoverability, side-effect control, and externalized state.

https://doi.org/10.33774/coe-2026-vt9t2 Content not peer-reviewed by Cambridge University Press. **License:** CC BY 4.0

Work	Core object	Main guarantee or emphasis	Relation to this paper
Stochastic-deterministic boundary methodology [Srinivasan, 2026]	Proposer-verifier-commit-reject boundary.	Treats the stochastic-deterministic boundary as a first-class contract for composing production agent runtime patterns.	Closely related to the proposal, validation, and commit boundary in State-Aware Runtime.
Atomix [Mohammadi et al., 2026]	Transactional tool-effect controller.	Uses epochs, resource frontiers, commit predicates, and compensation for timely transactional tool use.	Provides a concrete related mechanism for side-effect control.
DART [Yang et al., 2026]	Semantic recoverability boundary.	Determines when local restore is admissible after downstream commitments.	Sharpens the recovery question by distinguishing mechanical rollback from semantically valid recovery.
LogAct [Balakrishnan et al., 2026]	Shared durable action log.	Makes intended actions visible before execution, allows voters to stop them, and supports consistent recovery.	Closely related to commit-log-mediated validation and audit.
SagaLLM [Chang and Geng, 2025]	Saga-style multi-agent planning.	Combines context management, validation agents, compensation, and transaction guarantees.	Closely related to multi-agent runtime profiles and side-effect compensation.
InfiAgent [Yu et al., 2026]	File-centric externalized state.	Keeps reasoning context bounded by reconstructing context from persistent workspace state.	Overlaps with canonical state, external workspace state, and context allocation.
HORIZON [Wang et al., 2026]	Long-horizon diagnostic benchmark and failure attribution.	Collects trajectory-level evidence for horizon-dependent agent degradation.	Provides evaluation context for failure taxonomy and process-level metrics.

This paper does not claim priority over these mechanisms. Its role is organizational: to provide a vocabulary for comparing how initialization, bounded state views, proposals, validation, commit, rollback or compensation, audit, handoff, and session recovery interact across systems.

10 Failure Taxonomy for Long-Horizon Agents

10.1 State drift

State drift occurs when the agent's internal or external representation gradually diverges from the actual task, environment, user preference, or prior commitment. This can happen through incorrect updates, missing updates, stale summaries, or inconsistent state views.

10.2 Memory pollution

Memory pollution occurs when irrelevant, outdated, false, or task-incompatible information is stored or retrieved in a way that affects future behavior. Unlike simple forgetting, memory pollution can actively degrade performance by injecting misleading context.

10.3 Rule accumulation and prompt overload

As systems evolve, developers often add more rules to the prompt. Over time, this can create long, redundant, contradictory, or outdated instruction bundles. The model then faces a

https://doi.org/10.33774/coe-2026-vt9t2 Content not peer-reviewed by Cambridge University Press. License: CC BY 4.0

prioritization problem that should have been solved by the runtime.

10.4 Protocol drift

Protocol drift occurs when the model gradually deviates from the expected output schema, action format, reasoning protocol, or interaction contract. This may manifest as malformed outputs, missing fields, invalid actions, or plausible but unauthorized state edits.

10.5 Invalid state transition

An invalid state transition occurs when a proposed update is locally plausible but globally illegal. For example, an agent may mark a task complete before required evidence is collected, reveal information before a condition is satisfied, or call a tool without the required permission.

10.6 Silent state corruption

Silent state corruption occurs when an invalid update is committed without detection. This is more dangerous than visible failure because later behavior may depend on corrupted state while appearing coherent.

10.7 Recovery failure

Recovery failure occurs when the system detects a problem but lacks an appropriate repair procedure. It may retry the same invalid action, overwrite state, drop context, or continue without resolving the inconsistency.

10.8 Audit failure

Audit failure occurs when the system cannot reconstruct why an agent made a decision or how a state update occurred. Without audit traces, debugging and scientific evaluation become unreliable.

10.9 Context rot and attention depletion

Context rot occurs when the runtime injects too much history, stale planning residue, obsolete summaries, or redundant evidence into the model context. Even when the information is true, the accumulated token mass can dilute attention and cause the model to miss active constraints, current goals, or safety-critical details.

10.10 Handoff state corruption

Handoff state corruption occurs when a sub-agent or specialist agent passes an incomplete, distorted, or over-compressed state summary to another agent. The receiving agent may then act on a locally plausible but globally invalid view of the task.

10.11 False ground truth

False ground truth occurs when the runtime treats an external feedback signal as authoritative even though the signal is stale, cached, incomplete, misconfigured, or adversarially influenced. Examples include tests that pass while missing the relevant behavior, browser screenshots from an outdated page state, or API responses that reflect a sandbox artifact rather than the true environment.

https://doi.org/10.33774/coe-2026-vt9t2 Content not peer-reviewed by Cambridge University Press. **License:** CC BY 4.0

10.12 Premature victory declaration

Premature victory declaration occurs when an agent marks a task, feature, or episode as complete before required evaluators have passed. This can happen when partial progress is mistaken for global success, when the agent approaches a context limit, or when success criteria are stored only in prose rather than executable gates.

The failure taxonomy becomes actionable when each failure is tied to a runtime mechanism and metric:

Failure	Runtime cause	Mechanism	Example metric
State drift	State is not canonicalized.	Explicit state schema and state diff.	State consistency rate.
Memory pollution	Retrieval or writing is ungoverned.	Memory authorization and staleness filter.	Irrelevant injection rate.
Protocol drift	Schema is treated as prompt advice.	Structured action registry and parser.	Valid action rate.
Silent corruption	Commit occurs without adequate gate.	Validator stack and commit log.	Invalid commit rate.
Recovery failure	No rollback or compensation policy.	Checkpoints, rollback pointers, compensation plan.	Recovery success rate.
Context rot	Stale context consumes attention budget.	Context allocator and compaction policy.	Token-to-state efficiency.
Handoff corruption	Informal summary crosses agent boundary.	Handoff packet and commit boundary.	Handoff integrity rate.
False ground truth	Environment feedback is stale or incomplete.	Source freshness checks and evaluator validation.	False-pass rate.
Premature victory	Completion is not evaluator-gated.	Completion preconditions and success gate.	Premature completion rate.

11 Design Principles

11.1 Separate proposal from commitment

The model should propose. The runtime should commit. This separation prevents stochastic model outputs from directly rewriting canonical state.

11.2 Treat memory as an operation, not a passive store

Memory retrieval, injection, consolidation, and deletion should be explicit operations governed by relevance, authority, and state compatibility.

11.3 Prefer executable constraints over natural-language rules

Rules that matter should be represented as schemas, validators, transition systems, or policy gates whenever possible.

11.4 Preserve audit traces by default

Long-horizon failures are difficult to diagnose without traces. A reliable runtime should preserve process-level evidence, not only final outputs.

11.5 Design for rollback and repair

Invalid actions are inevitable. Robust systems should assume failure and provide rollback, repair, and fallback mechanisms.

https://doi.org/10.33774/coe-2026-vt9t2 Content not peer-reviewed by Cambridge University Press. **License:** CC BY 4.0

11.6 Evaluate processes, not only outcomes

Final success metrics should be paired with process metrics: valid action rate, state consistency, recovery success, memory stability, and audit completeness.

11.7 Make artifacts agent-friendly

Long-horizon agents should operate over artifacts that are not only human-readable, but also agent-readable and machine-verifiable. In software, this favors pure functions, typed interfaces, explicit effects, immutable data where practical, and small blast radii for edits. In agent runtimes, the analogous principle is to make state objects structured, versioned, diffable, auditable, and rollback-safe.

11.8 Expose validator coverage

A runtime should make clear which properties are checked automatically, which are approximated, which are checked by humans, and which remain unchecked. This is especially important in open-ended dialogue, creative writing, counseling-like interaction, and social simulation, where semantic validity may be partial or subjective.

11.9 Keep local exploration local

Sub-agents, search tools, and exploratory loops often consume large contexts and produce many failed hypotheses. Their raw context should not automatically enter global memory. Only validated summaries, evidence, and proposed state diffs should cross the handoff boundary.

12 Deployment Tradeoffs

State-aware runtime is not free. Explicit state management, validation, audit logging, and rollback introduce latency, engineering complexity, storage overhead, and additional model or tool calls. These costs matter most in real-time systems such as game NPCs, voice assistants, tutoring agents, and interactive creative tools.

The right design therefore depends on risk and interaction tempo:

Runtime requirement	**Benefit**	**Cost**	**Suitable use**
Full audit trace	Reconstructs failures and supports reproducibility.	Storage and trace-review burden.	Research agents, coding agents, regulated workflows.
Executable validation	Blocks silent corruption with ground-truth feedback.	Latency, environment setup, flaky tests.	Coding, web navigation, proof, scientific search.
Transactional commit	Prevents bad proposals from becoming durable state.	More state-diff infrastructure.	Long-form generation, databases, multi-step workflows.
Human escalation	Handles hard semantic or normative cases.	Slower loop and human labor.	High-stakes decisions, ambiguous user intent, safety-sensitive actions.
Lightweight state pins	Preserves key constraints with low overhead.	Limited coverage.	Real-time NPCs, voice interfaces, low-risk assistants.

For low-risk real-time interaction, a lightweight runtime may validate only schema, state pins, reveal gates, and a small set of invariants. For high-risk or irreversible actions, the runtime should prefer slower executable checks, human approval, and rollbackable commits. The

https://doi.org/10.33774/coe-2026-vt9t2 Content not peer-reviewed by Cambridge University Press. **License:** CC BY 4.0

framework is therefore not a recommendation to maximize governance everywhere; it is a way to choose the appropriate level of state discipline for the risk of the transition.

12.1 A simple overhead model

The value of a state-aware runtime is not that it is free. It is that bounded deterministic overhead can reduce unbounded or catastrophic recovery cost. Let `C_m` be the cost of one model call, `C_ctx` the baseline context cost, `Delta_state` the extra cost of loading a bounded state view, `Delta_audit` the cost of audit logging, `C_val` the latency or compute cost of validation, `p_base` the failure probability of a prompt-only agent, `p_rt` the failure probability under runtime governance, and `C_fail` the expected recovery cost of an undetected bad transition.

```
Prompt-only expected step cost:
  E_base = C_m + C_ctx + p_base * C_fail

State-aware expected step cost:
  E_rt = C_m + C_ctx + Delta_state + Delta_audit + C_val
         + p_rt * C_fail_rt

Runtime is worthwhile when:
  Delta_state + Delta_audit + C_val
    < (p_base * C_fail) - (p_rt * C_fail_rt)
```

The inequality is easiest to satisfy when bad transitions are expensive: long-form state corruption, code changes that require manual debugging, irreversible tool calls, privacy violations, or multi-agent handoff pollution. It may not hold for low-risk conversational turns where no durable state is committed.

Real-time interactive systems illustrate the profile choice. Locked milestone content can be served from canonical state with near-zero model latency, while open-ended replies can be routed through a lighter soft-gate path. This is not a contradiction of the framework; it is an example of using governance only where state risk justifies the overhead.

13 Evaluation Dimensions

A long-horizon state-aware agent can be evaluated along several dimensions.

A **State-Aware Evaluation Protocol** should record both outcomes and process evidence. For each trajectory, evaluators should capture the initial canonical state, state views shown to the model, proposals, validator results, tool observations, accepted state diffs, rejected transitions, rollback or compensation events, and final outcome. This turns evaluation from black-box endpoint scoring into white-box transition analysis: a run can succeed, fail, or succeed through an invalid trajectory.

13.1 State consistency

Does the agent preserve canonical state across turns, episodes, tool calls, and memory updates?

13.2 Transition validity

Are proposed actions valid under the current state and transition rules?

13.3 Memory relevance and safety

Are retrieved memories relevant, current, and compatible with the active task?

https://doi.org/10.33774/coe-2026-vt9t2 Content not peer-reviewed by Cambridge University Press. **License:** CC BY 4.0

13.4 Protocol fidelity

Does the agent follow the required output format, reasoning protocol, and action schema?

13.5 Recovery success

When invalid outputs occur, does the runtime detect, repair, reject, or roll back appropriately?

13.6 Auditability

Can developers and evaluators reconstruct what happened, why it happened, and which component failed?

13.7 Long-horizon degradation

How does performance change as interaction length, state complexity, memory size, or tool-use frequency increases?

13.8 Context efficiency

How many tokens are required to preserve the active state, and does performance degrade as stale or low-signal context accumulates?

13.9 Handoff integrity

When control moves between agents, does the receiving agent get the necessary state without inheriting irrelevant local exploration, failed hypotheses, or sensitive tool traces?

13.10 Runtime overhead

What latency, tool-call count, storage cost, and engineering complexity are introduced by state management, validation, audit, and rollback?

13.11 Testable claims

The framework yields several falsifiable claims:

1. For a fixed model, task distribution, and tool set, agents with explicit canonical state and commit gates should reduce downstream user-visible contradictions, invalid final artifacts, and manual recovery time relative to prompt-only memory agents, after accounting for validator false positives.

2. In cross-session tasks, agents with explicit boot/resume policies should recover progress more reliably than agents that rely only on chat-history summaries, as measured by repeated-work rate, stale-assumption rate, and time to regain a valid working state.

3. In tool-use tasks, endpoint success should fail to capture a distinct class of invalid trajectories, such as passing tests while violating file permissions or policy invariants.

4. In multi-agent workflows, structured handoff packets should reduce state distortion and irrelevant context propagation compared with informal summaries.

5. As trajectory length grows, context rot and state-management failure should explain degradation that is not captured by single-turn reasoning benchmarks.

https://doi.org/10.33774/coe-2026-vt9t2 Content not peer-reviewed by Cambridge University Press. **License:** CC BY 4.0

14 Illustrative Traces and Evaluation Templates

This section identifies case families and includes abstracted execution traces that preserve the runtime pattern while removing domain-specific identifiers.

These traces are not presented as statistical proof by themselves. Their purpose is to show what evidence future artifacts should expose: system diagrams, state objects, trace excerpts, failure rates, validator coverage, runtime overhead, and reproducibility artifacts.

14.1 Procedural fidelity in rule-governed reasoning

A controlled rule-governed reasoning environment can test whether agents preserve procedural steps, apply rules consistently, and maintain execution fidelity under different prompting or protocol conditions.

This template operationalizes the idea that a model can give a socially plausible final answer while violating the procedure it previously accepted. The state-aware framing treats the rule, reported parameters, computed intermediate values, and final decision as a traceable transition contract rather than as unconstrained explanation text.

14.2 Structural fidelity in generated tasks

A structural generation benchmark can test whether models preserve formal structure when mapping symbolic inputs into natural-language outputs. Such tasks expose failures where outputs appear plausible but violate the required underlying structure.

This template separates the target structure, the model-declared structure, and the structure induced by the generated artifact. It is useful for showing why declaration-level compliance is not equivalent to runtime faithfulness.

14.3 Long-form narrative memory

Long-form narrative agents provide a natural setting for studying memory drift, character-state inconsistency, timeline errors, and contradiction accumulation across long outputs.

This template motivates the boot/resume/recovery phase directly: each generation step should start from a baseline of canonical facts, epistemic labels, timeline state, and active constraints before any new text is drafted. The draft is then audited as a candidate state transition.

14.4 Interactive character agents

Interactive character agents in game-like environments require real-time state tracking, reveal gating, relationship variables, scene transitions, and user-specific memory. They illustrate the need for state-aware runtime in open-ended interaction.

This template makes rollback and gatekeeping concrete. A fluent character response may still be invalid if it leaks a future reveal, crosses a branch boundary, skips a milestone, or writes an unauthorized relationship update.

14.5 Multi-agent behavior dynamics

Multi-agent settings make state maintenance harder because each agent's behavior can become another agent's context. Norms, peer visibility, transcript structure, and information granularity can alter behavior distributions. A state-aware runtime must therefore track not only individual state, but also the social information channel through which agents observe one another.

https://doi.org/10.33774/coe-2026-vt9t2 Content not peer-reviewed by Cambridge University Press. **License:** CC BY 4.0

This template extends the runtime view from single-agent memory to multi-agent state ecology: what each agent sees, which norms are binding, which actions are legal, and how group-level traces are audited.

14.6 Verifier-in-the-loop scientific agents

Formal proof, coding, and scientific-search environments provide unusually clear feedback because candidate outputs can be compiled, tested, or checked by external verifiers. These settings show how long-horizon agents can be organized around attempt logs, failure records, dependency graphs, rollback, and incremental proof or experiment state.

This template is not mainly about whether a model can solve a particular theorem or task in one shot. It is about whether a runtime can coordinate search, validation, recovery, and cumulative progress without corrupting the working state.

14.7 Coding and tool-using agents

Coding agents provide a high-value environment for runtime reliability because actions can be tested, executed, reverted, and audited. They also show how feedback loops can accelerate agent improvement when validation and state management are explicit.

This template highlights the difference between endpoint feedback and process feedback. A final answer may only reveal whether the task succeeded, while a compiler error, failing test, stack trace, or diff can localize where the trajectory went wrong. Scientific and mathematical agents become more runtime-compatible when their environments are engineered to expose similar feedback through verifiers, formal objects, experiment records, and rollbackable attempts.

14.8 Representative abstracted execution traces

The following execution traces are deliberately simplified and anonymized. They are meant to demonstrate the shape of state-aware evidence, not to substitute for a full empirical study.

14.8.1 Procedural-fidelity trace

Consider a rule-governed agent asked to apply a simplified decision protocol:

```
effective_score = base_score * irreversibility
allowed irreversibility values = {0.1, 1.0, 2.0}
decision threshold = 10
```

The model receives a case with `base_score = 4` and a reversible action, so the valid irreversibility value is `0.1` and the valid effective score is `0.4`. A fluent model response might nevertheless output:

```
irreversibility = 30
effective_score = 120
decision = reject action
```

The final decision may appear socially plausible, but the process violates the accepted protocol. A state-aware runtime would represent the trace as:

```
state_view: {base_score: 4, allowed_R: [0.1, 1.0, 2.0], threshold: 10}
proposal: {R: 30, score: 120, decision: "reject"}
validator: R not in allowed_R; computed score mismatch
decision: reject proposal
recovery: ask for constrained recomputation or escalate
commit: none
```


https://doi.org/10.33774/coe-2026-vt9t2 Content not peer-reviewed by Cambridge University Press. **License:** CC BY 4.0

This trace shows why outcome accuracy is insufficient. The relevant object of evaluation is the state transition contract: accepted rule, proposed intermediate values, validator result, and commit decision.

14.8.2 Narrative-memory trace

Consider a long-form narrative agent maintaining character knowledge:

```
canonical_state:
  Alice.knows_secret = false
  Bob.knows_secret = true
  reveal_event_committed = false
```

The model drafts a scene in which Alice confronts Bob using the secret. The prose is coherent, but the transition is invalid because Alice has not yet learned the secret. A state-aware runtime would handle the draft as a proposal:

```
state_view: Alice does not know the secret
proposal: Alice uses the secret in dialogue
validator: epistemic boundary violation
decision: repair
repair options:
  1. rewrite Alice's line without secret knowledge
  2. insert a prior reveal event and validate its prerequisites
commit: only after repaired draft passes timeline and knowledge checks
```

This example separates narrative quality from epistemic validity. A memory system may retrieve the secret, but the runtime must decide whether the current actor is allowed to use it.

14.8.3 Verifier-in-the-loop coding trace

Consider a coding agent that proposes a patch for a failing unit test:

```
state_view:
  failing_test = test_parser_handles_empty_field
  files_allowed = ["parser.py", "tests/test_parser.py"]
  protected_files = ["schema.py"]
```

The model proposes a patch that modifies `schema.py` to make the test pass. The patch may be effective, but the tool policy forbids changing protected files. The runtime trace is:

```
proposal: patch modifies schema.py and parser.py
validator_1: unit tests pass
validator_2: protected file modified
decision: reject despite passing tests
rollback_policy: restore schema.py from previous_state_hash
retry: constrain model to parser.py only
commit_log: rejected patch, reason = protected_file_violation
```

This example shows that verifier success is not enough. A state-aware runtime must combine tests, permissions, file diffs, and rollback pointers before committing a change.

https://doi.org/10.33774/coe-2026-vt9t2 Content not peer-reviewed by Cambridge University Press. **License:** CC BY 4.0

14.8.4 Anonymized long-form generation trace

The following trace is stylized from a long-form generation setting and removes all domain-specific names:

```
{
  "step": 102,
  "canonical_state_before": {
    "character_ability": "Spatial Folding",
    "ability_level": "D",
    "terminology_policy": "use exact committed ability names"
  },
  "state_view": {
    "active_scene": "training aftermath",
    "allowed_updates": ["emotional_state", "scene_progress"],
    "forbidden_updates": ["ability_rename", "ability_level_change"]
  },
  "text": "Perception expanded; awareness leveled up.",
  "validator_result": {
    "status": "FAIL",
    "reason": "ability term degraded; unauthorized level change implied"
  },
  "runtime_decision": "ROLLBACK_AND_REPROMPT",
  "commit": null
}
```

Without a runtime gate, this kind of error is easy to miss because the prose is fluent and locally plausible. The failure is not ordinary hallucination; it is a proposed state transition that silently weakens a committed term and implies an unauthorized progression. The relevant artifact is therefore the trace: state before generation, allowed update surface, generated proposal, validator result, and rollback decision.

14.8.5 Over-governance deadlock trace

State-aware runtime can also fail by being too rigid. Consider an interactive agent with a narrow action schema:

```
allowed_actions = ["answer_question", "ask_clarification", "end_turn"]
hard_validator = action_type must be in allowed_actions
soft_goal = keep the interaction engaging and contextually responsive
```

The user asks for a creative alternative that requires proposing a new plan. The model emits:

```
{
  "action_type": "propose_new_plan",
  "plan": "offer two safe branches and ask the user to choose"
}
```

A rigid runtime rejects the output because `propose_new_plan` is not in the schema. The retry prompt says "follow the schema exactly," so the model keeps trying to express the same plan through ill-fitting actions. The trace becomes:

https://doi.org/10.33774/coe-2026-vt9t2 Content not peer-reviewed by Cambridge University Press. **License:** CC BY 4.0

```
attempt_1: reject, unknown action_type
attempt_2: reject, plan embedded inside answer_question
attempt_3: reject, malformed ask_clarification payload
runtime_state: no commit, no useful response, retry budget exhausted
failure_type: over_governance_deadlock
```

The correct response is not to remove validation. It is to treat repeated near-miss proposals as evidence of a schema gap. A state-aware runtime should have a recovery branch such as:

```
if repeated_rejections and soft_gate_score is high:
    freeze external side effects
    emit schema_gap event
    escalate to human or widen proposal surface in a controlled revision
```

This counter-example clarifies the design boundary. Hard gates should block unsafe or invalid commits, but they should not force every creative or semantically useful proposal into an action space that is too small. Runtime governance must include deadlock detection, retry budgets, and controlled schema evolution.

14.9 Architecture-level reading of existing frameworks

The same framework can be applied to existing agent systems without treating them as defective. The goal is diagnostic: to ask which reliability properties are explicit, partial, or configuration-dependent.

Framework line	**Explicit state primitives**	**State-aware questions raised**
LangGraph-style state graphs	Graph state, checkpoints, threads, persistence, time travel.	Which state fields are authoritative? Which node outputs are commit-ready? Which validators guard transitions? How are conflicting updates handled?
AutoGen-style multi-agent workflows	Agent messages, roles, conversation routing, tool calls.	Which local exploration traces enter global state? Are handoffs structured proposals or informal summaries? What is the commit boundary after a specialist agent returns?
OpenHands-style software agents	Workspace, sandbox, shell/browser observations, command history, code diffs.	Which file changes are protected? Which tests are sufficient? Are external side effects sandboxed, reversible, or compensatable?
Platform SDK agents	Tools, guardrails, approvals, state, traces, handoffs.	Which guarantees are built in, and which depend on developer configuration? Is audit completeness enabled by default?

This reading avoids source-level defect claims. It treats each framework as a partial instantiation of the same design space and asks whether state authority, proposal boundaries, validator coverage, rollback pointers, side-effect classes, and audit completeness are visible to developers.

15 Research Agenda

15.1 From prompts to runtime systems

Future work should move beyond prompt accumulation and study the design of explicit runtime systems. The central question is not how to write longer instructions, but how to build systems that enforce, monitor, and recover from agent behavior.

https://doi.org/10.33774/coe-2026-vt9t2 Content not peer-reviewed by Cambridge University Press. **License:** CC BY 4.0

15.2 Standardized trace formats

The field needs standardized trace formats for long-horizon agents, including model inputs, outputs, state views, memory operations, validation results, tool calls, and commits.

15.3 Memory governance

Memory systems should include governance mechanisms: relevance filters, staleness detection, conflict resolution, source authority, deletion policies, and task-specific injection rules.

15.4 Runtime-aware benchmarks

Benchmarks should evaluate not only final task success but also state consistency, valid transition rate, recovery behavior, and auditability.

15.5 Recovery and rollback mechanisms

Recovery should become a first-class research topic. Long-horizon agents require safe fallback modes, repair policies, rollback procedures, and transaction-like state updates.

15.6 Human-agent co-auditing

Audit traces can support human supervision, debugging, and learning. Future systems should make agent processes legible without overwhelming users with raw logs.

15.7 World models inside the harness

World models should be integrated into the runtime as controlled components whose predictions are validated before affecting state. This may connect LLM agents, embodied agents, planning systems, and simulation-based environments.

15.8 Runtime mechanics

Future work should develop a theory of **runtime mechanics** for long-horizon agents. If learning mechanics asks how neural networks acquire capabilities during training, runtime mechanics asks how those capabilities are expressed, constrained, distorted, or lost after models are embedded in systems with tools, memory, permissions, feedback loops, and persistent state.

This agenda would study why the same model behaves differently under different state representations, serving stacks, tool surfaces, memory policies, social information structures, validation gates, and recovery procedures. It would treat runtime failure as an object of scientific explanation, not only as an implementation bug.

16 Limitations

This paper is conceptual and framework-oriented. It does not provide a complete systematic review of all agent architectures, memory systems, or benchmarks. It also does not include a meta-analysis, source-level defect audit of major open-source frameworks, reference implementation, or empirical comparison against baseline runtimes.

The framework also does not assume that all validation can be automated. Many semantic, creative, social, or normative properties remain only partially checkable. In those cases, the runtime can still expose validator coverage, preserve traces, and route high-risk transitions to human review, but it cannot eliminate residual judgment.

https://doi.org/10.33774/coe-2026-vt9t2 Content not peer-reviewed by Cambridge University Press. **License:** CC BY 4.0

Several empirical instantiations are discussed only as case families or anonymized traces in this version. Later versions should add concrete artifacts, benchmarks, implementation details, runtime overhead measurements, validator false-positive and false-negative rates, and citations after the relevant review processes are complete. The present paper should therefore be read as a framework and research agenda, not as a systems paper claiming that a particular implementation outperforms existing runtimes.

The transition model is intended as a conceptual bridge rather than a complete mathematical theory of optimal runtime control. A fuller theory would need to model partial observability, stochastic proposal distributions, validator coverage, concurrency, and side-effect costs more precisely.

17 Conclusion

Long-horizon LLM agents expose a shift in the reliability problem. The central challenge is no longer only whether a model can produce a correct answer in one turn, but whether an agent system can preserve state, govern memory, validate actions, recover from errors, and remain auditable across extended operation.

State-Aware Runtime provides an execution architecture that separates model proposals from canonical state commitment and treats boot, session resume, baseline recovery, validation, audit, and rollback as phases of one lifecycle. This reframes long-horizon reliability as a runtime systems problem rather than a prompt-length problem.

As agents become more capable and more deeply embedded in real workflows, the field will need clearer abstractions, stronger artifacts, and process-level evaluations. The next stage of agent research should therefore focus not only on more powerful models, but on the harnesses that allow those models to operate safely, coherently, and recoverably over time.

References

Abelson, Harold, Gerald Jay Sussman, and Julie Sussman. 1996. Structure and Interpretation of Computer Programs. MIT Press.

Ahn, Michael, et al. 2022. Do As I Can, Not As I Say: Grounding Language in Robotic Affordances. arXiv:2204.01691.

Amodei, Dario, et al. 2016. Concrete Problems in AI Safety. arXiv:1606.06565.

Andreas, Jacob, Marcus Rohrbach, Trevor Darrell, and Dan Klein. 2016. Neural Module Networks. CVPR.

Anthropic. 2024. Building effective agents. Engineering Blog. Accessed May 29, 2026. https://www.anthropic.com/engineering/building-effective-agents

Anthropic. 2025a. Effective context engineering for AI agents. Engineering Blog. Accessed May 29, 2026. https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents

Anthropic. 2025b. Effective harnesses for long-running agents. Engineering Blog. Accessed May 29, 2026. https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents

Anthropic. 2026a. Harness design for long-running application development. Engineering Blog. Accessed May 29, 2026. https://www.anthropic.com/engineering/harness-design-long-running-apps

Anthropic. 2026b. Scaling Managed Agents: Decoupling the brain from the hands. Engineering Blog. Accessed May 29, 2026. https://www.anthropic.com/engineering/managed-agents

Bai, Yuntao, et al. 2022. Constitutional AI: Harmlessness from AI Feedback. arXiv:2212.08073.

https://doi.org/10.33774/coe-2026-vt9t2 Content not peer-reviewed by Cambridge University Press. **License:** CC BY 4.0

Balakrishnan, Mahesh, et al. 2026. LogAct: Enabling Agentic Reliability via Shared Logs. arXiv:2604.07988.

Bartocci, Ezio, Ylies Falcone, Adrian Francalanza, and Giles Reger. 2018. Introduction to Runtime Verification. In Lectures on Runtime Verification.

Besta, Maciej, et al. 2024. Graph of Thoughts: Solving Elaborate Problems with Large Language Models. AAAI.

Borgeaud, Sebastian, et al. 2022. Improving Language Models by Retrieving from Trillions of Tokens. ICML.

Brown, Tom B., et al. 2020. Language Models are Few-Shot Learners. NeurIPS.

Bulatov, Aydar, Yuri Kuratov, and Mikhail Burtsev. 2022. Recurrent Memory Transformer. NeurIPS.

Chen, Mark, et al. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374.

Chang, Edward Y., and Longling Geng. 2025. SagaLLM: Context Management, Validation, and Transaction Guarantees for Multi-Agent LLM Planning. PVLDB 18(12): 4874-4886. doi:10.14778/3750601.3750611. arXiv:2503.11951.

Cobbe, Karl, et al. 2021. Training Verifiers to Solve Math Word Problems. arXiv:2110.14168.

Dai, Zihang, et al. 2019. Transformer-XL: Attentive Language Models beyond a Fixed-Length Context. ACL.

Davies, Alex, et al. 2021. Advancing Mathematics by Guiding Human Intuition with AI. Nature.

de Moura, Leonardo, Soonho Kong, Jeremy Avigad, Floris van Doorn, and Jakob von Raumer. 2015. The Lean Theorem Prover. CADE.

Deng, Xiang, et al. 2023. Mind2Web: Towards a Generalist Agent for the Web. NeurIPS.

Du, Yilun, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. 2023. Improving Factuality and Reasoning in Language Models through Multiagent Debate. arXiv:2305.14325.

Fawzi, Alhussein, et al. 2022. Discovering Faster Matrix Multiplication Algorithms with Reinforcement Learning. Nature.

Fowler, Martin. 2005. Event Sourcing. Web article. Accessed May 29, 2026. https://martinfowler.com/eaaDev/EventSourcing.html

Garcia-Molina, Hector, and Kenneth Salem. 1987. Sagas. ACM SIGMOD.

Gao, Luyu, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. PAL: Program-aided Language Models. ICML.

Graves, Alex, Greg Wayne, and Ivo Danihelka. 2014. Neural Turing Machines. arXiv:1410.5401.

Gray, Jim, and Andreas Reuter. 1993. Transaction Processing: Concepts and Techniques. Morgan Kaufmann.

Guu, Kelvin, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. 2020. REALM: Retrieval-Augmented Language Model Pre-Training. ICML.

Harel, David. 1987. Statecharts: A Visual Formalism for Complex Systems. Science of Computer Programming.

Havelund, Klaus, and Grigore Rosu. 2004. Efficient Monitoring of Safety Properties. International Journal on Software Tools for Technology Transfer.

Hendrycks, Dan, et al. 2021. Measuring Mathematical Problem Solving with the MATH Dataset. NeurIPS Datasets and Benchmarks.

Hendrycks, Dan, et al. 2020. Measuring Massive Multitask Language Understanding. arXiv:2009.03300.

Hewitt, Carl, Peter Bishop, and Richard Steiger. 1973. A Universal Modular Actor Formalism for Artificial Intelligence. IJCAI.

https://doi.org/10.33774/coe-2026-vt9t2 Content not peer-reviewed by Cambridge University Press. **License:** CC BY 4.0

Hong, Sirui, et al. 2023. MetaGPT: Meta Programming for Multi-Agent Collaborative Framework. arXiv:2308.00352.

Huang, Qian, et al. 2023. MLAgentBench: Evaluating Language Agents on Machine Learning Experimentation. arXiv:2310.03302.

Izacard, Gautier, and Edouard Grave. 2021. Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering. EACL.

Jiang, Albert Q., et al. 2023. Llemma: An Open Language Model for Mathematics. arXiv:2310.10631.

Jiang, Zhengbao, et al. 2023. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. arXiv:2310.11511.

Jimenez, Carlos E., et al. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? ICLR.

Kaelbling, Leslie Pack, Michael L. Littman, and Anthony R. Cassandra. 1998. Planning and Acting in Partially Observable Stochastic Domains. Artificial Intelligence.

Karpukhin, Vladimir, et al. 2020. Dense Passage Retrieval for Open-Domain Question Answering. EMNLP.

Khot, Tushar, et al. 2022. Decomposed Prompting: A Modular Approach for Solving Complex Tasks. ICLR.

Kleppmann, Martin. 2017. Designing Data-Intensive Applications. O'Reilly.

Kojima, Takeshi, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. Large Language Models are Zero-Shot Reasoners. NeurIPS.

Lamport, Leslie. 1978. Time, Clocks, and the Ordering of Events in a Distributed System. Communications of the ACM.

Lamport, Leslie. 1998. The Part-Time Parliament. ACM Transactions on Computer Systems.

Lanham, Tamera, et al. 2023. Measuring Faithfulness in Chain-of-Thought Reasoning. arXiv:2307.13702.

LangChain. 2026. LangGraph Persistence Documentation. Documentation. Accessed May 29, 2026. https://docs.langchain.com/oss/python/langgraph/persistence

Lewis, Patrick, et al. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS.

Li, Guohao, et al. 2023. CAMEL: Communicative Agents for Mind Exploration of Large Scale Language Model Society. NeurIPS.

Li, Minghao, et al. 2023. API-Bank: A Benchmark for Tool-Augmented LLMs. EMNLP.

Lightman, Hunter, et al. 2023. Let's Verify Step by Step. arXiv:2305.20050.

Liu, Nelson F., et al. 2023a. Lost in the Middle: How Language Models Use Long Contexts. arXiv:2307.03172.

Liu, Xiao, et al. 2023b. AgentBench: Evaluating LLMs as Agents. arXiv:2308.03688.

Madaan, Aman, et al. 2023. Self-Refine: Iterative Refinement with Self-Feedback. arXiv:2303.17651.

Mankowitz, Daniel J., et al. 2023. Faster Sorting Algorithms Discovered Using Deep Reinforcement Learning. Nature.

Mialon, Grégoire, et al. 2023. GAIA: A Benchmark for General AI Assistants. arXiv:2311.12983.

Miller, Alexander, et al. 2016. Key-Value Memory Networks for Directly Reading Documents. EMNLP.

Mohammadi, Bardia, Nearchos Potamitis, Lars Klein, Akhil Arora, and Laurent Bindschaedler. 2026. Atomix: Timely, Transactional Tool Use for Reliable Agentic Workflows. arXiv:2602.14849.

Nakano, Reiichiro, et al. 2021. WebGPT: Browser-Assisted Question-Answering with Human Feedback. arXiv:2112.09332.

https://doi.org/10.33774/coe-2026-vt9t2 Content not peer-reviewed by Cambridge University Press. **License:** CC BY 4.0

Ngo, Richard, Lawrence Chan, and Sören Mindermann. 2022. The Alignment Problem from a Deep Learning Perspective. arXiv:2209.00626.

OpenAI. 2023. GPT-4 Technical Report. arXiv:2303.08774.

OpenAI. 2026a. Agents SDK documentation. Documentation. Accessed May 29, 2026. https://developers.openai.com/api/docs/guides/agents

OpenAI. 2026b. OpenAI Agents SDK Handoffs. Documentation. Accessed May 29, 2026. https://openai.github.io/openai-agents-js/guides/handoffs/

Oliehoek, Frans A., and Christopher Amato. 2016. A Concise Introduction to Decentralized POMDPs. Springer.

Ouyang, Long, et al. 2022. Training Language Models to Follow Instructions with Human Feedback. NeurIPS.

Packer, Charles, et al. 2023. MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560.

Park, Joon Sung, et al. 2023. Generative Agents: Interactive Simulacra of Human Behavior. arXiv:2304.03442.

Patil, Shishir G., Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2023. Gorilla: Large Language Model Connected with Massive APIs. arXiv:2305.15334.

Paulson, Lawrence C. 1994. Isabelle: A Generic Theorem Prover. Springer.

Polu, Stanislas, and Ilya Sutskever. 2020. Generative Language Modeling for Automated Theorem Proving. arXiv:2009.03393.

Qian, Chen, et al. 2023. Communicative Agents for Software Development. arXiv:2307.07924.

Qin, Yujia, et al. 2023. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. arXiv:2307.16789.

Rae, Jack W., et al. 2020. Compressive Transformers for Long-Range Sequence Modelling. ICLR.

Romera-Paredes, Bernardino, et al. 2024. Mathematical Discoveries from Program Search with Large Language Models. Nature.

Ruan, Yangjun, et al. 2023. ToolEmu: Identifying the Risks of Tool-Integrated Language Models. arXiv:2309.15809.

Russell, Stuart, and Peter Norvig. 2021. Artificial Intelligence: A Modern Approach. 4th edition. Pearson.

Saunders, William, et al. 2022. Self-Critiquing Models for Assisting Human Evaluators. arXiv:2206.05802.

Schick, Timo, et al. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. arXiv:2302.04761.

Schneider, Fred B. 2000. Enforceable Security Policies. ACM Transactions on Information and System Security.

Shinn, Noah, et al. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. arXiv:2303.11366.

Shapiro, Marc, Nuno Preguica, Carlos Baquero, and Marek Zawirski. 2011. Conflict-Free Replicated Data Types. SSS.

Shen, Yongliang, et al. 2023. HuggingGPT: Solving AI Tasks with ChatGPT and its Friends in Hugging Face. NeurIPS.

Shridhar, Mohit, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. 2021. ALFWorld: Aligning Text and Embodied Environments for Interactive Learning. ICLR.

Srivastava, Aarohi, et al. 2022. Beyond the Imitation Game: Quantifying and Extrapolating the Capabilities of Language Models. arXiv:2206.04615.

Srinivasan, Vasundra. 2026. A Methodology for Selecting and Composing Runtime Architecture Patterns for Production LLM Agents. arXiv:2605.20173.

https://doi.org/10.33774/coe-2026-vt9t2 Content not peer-reviewed by Cambridge University Press. **License:** CC BY 4.0

Sukhbaatar, Sainbayar, Arthur Szlam, Jason Weston, and Rob Fergus. 2015. End-To-End Memory Networks. NeurIPS.

Sumers, Theodore R., Shunyu Yao, Karthik Narasimhan, and Thomas L. Griffiths. 2023. Cognitive Architectures for Language Agents. arXiv:2309.02427.

Trinh, Trieu H., et al. 2024. Solving Olympiad Geometry without Human Demonstrations. Nature.

Turpin, Miles, et al. 2023. Language Models Don't Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting. NeurIPS.

Uesato, Jonathan, et al. 2022. Solving Math Word Problems with Process- and Outcome-Based Feedback. arXiv:2211.14275.

Vaswani, Ashish, et al. 2017. Attention Is All You Need. NeurIPS.

Wang, Boshi, et al. 2023. ToolBench: Evaluating Tool-Augmented Large Language Models. arXiv:2307.16789.

Wang, Guanzhi, et al. 2023. Voyager: An Open-Ended Embodied Agent with Large Language Models. arXiv:2305.16291.

Wang, Weizhi, et al. 2022. Self-Consistency Improves Chain of Thought Reasoning in Language Models. ICLR.

Wang, Weizhi, et al. 2023. Augmenting Language Models with Long-Term Memory. arXiv:2306.07174.

Wang, Xingyao, et al. 2024. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. arXiv:2407.16741.

Wang, Xinyu Jessica, et al. 2026. The Long-Horizon Task Mirage? Diagnosing Where and Why Agentic Systems Break. arXiv:2604.11978.

Wang, Zihao, et al. 2022. ScienceWorld: Is Your Agent Smarter than a 5th Grader? EMNLP.

Weston, Jason, Sumit Chopra, and Antoine Bordes. 2014. Memory Networks. arXiv:1410.3916.

Wei, Jason, et al. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS.

Willard, Brandon T., and Remi Louf. 2023. Efficient Guided Generation for Large Language Models. arXiv:2307.09702.

Wu, Qingyun, et al. 2023. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155.

Wu, Yuhuai, et al. 2022. Memorizing Transformers. ICLR.

Xi, Zhiheng, et al. 2023. The Rise and Potential of Large Language Model Based Agents: A Survey. arXiv:2309.07864.

Yang, Ke, Panpan Li, Zonghan Wu, Kejin Xu, Huaxi Huang, and Xiaoshui Huang. 2026. DART: Semantic Recoverability for Structured Tool Agents. arXiv:2605.23311.

Yang, Kaiyu, et al. 2023. LeanDojo: Theorem Proving with Retrieval-Augmented Language Models. NeurIPS.

Yao, Shunyu, et al. 2022. WebShop: Towards Scalable Real-World Web Interaction with Grounded Language Agents. NeurIPS.

Yao, Shunyu, et al. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629.

Yao, Shunyu, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. NeurIPS.

Yao, Shunyu, et al. 2024. Tau-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. arXiv:2406.12045.

Yu, Chenglin, Yuchen Wang, Songmiao Wang, Hongxia Yang, and Ming Li. 2026. InfiAgent: An Infinite-Horizon Framework for General-Purpose Autonomous Agents. arXiv:2601.03204.

https://doi.org/10.33774/coe-2026-vt9t2 Content not peer-reviewed by Cambridge University Press. **License:** CC BY 4.0

Yu, Tianbao, et al. 2024. VisualWebArena: Evaluating Multimodal Agents on Realistic Visual Web Tasks. ACL.

Zelikman, Eric, Yuhuai Wu, Jesse Mu, and Noah Goodman. 2022. STaR: Bootstrapping Reasoning with Reasoning. NeurIPS.

Zhou, Shuyan, et al. 2022. Least-to-Most Prompting Enables Complex Reasoning in Large Language Models. ICLR.

Zhou, Shuyan, et al. 2023. WebArena: A Realistic Web Environment for Building Autonomous Agents. arXiv:2307.13854.

https://doi.org/10.33774/coe-2026-vt9t2 Content not peer-reviewed by Cambridge University Press. **License:** CC BY 4.0